Memento PL/SQL

Exemple simple.

```
DECLARE
    vidfilm NUMBER(3); --types admis : types SQL (Varchar2(20), number(P,q)...), Boolean
    vpays film.paysfilm%TYPE; --vpays a le meme type que paysfilm
    vfilm film%ROWTYPE; --vfilm a autant de colonnes que film
    nbnotes NUMBER(3) := 0; --variable declaree et initialisee
    moy NUMBER(4, 2);
BEGIN
    vidfilm := :cequevousvoulez; --affectation et lecture au clavier
    SELECT * INTO vfilm FROM film WHERE idfilm = vidfilm;
    SELECT count(*), avg(note) INTO nbnotes, moy FROM notation WHERE idfilm = vidfilm;
    vpays := vfilm.paysfilm;
    IF vpays = 'France' THEN
        dbms_output.put_line(vfilm.titre||' est francais, cocorico');
    ELSIF vpays = 'USA' THEN
        dbms_output.put_line(vfilm.titre||' est americain, pas cocorico');
    ELSE
        dbms_output.put_line(vfilm.titre||' n''est pas francais, pas cocorico');
    END IF;
    dbms_output.put_line('Moyenne :'||moy||' nb de notes : '||nbnotes);
END;
--boucles FOR i IN [REVERSE] 1..N LOOP ... END LOOP;
-- WHILE condition LOOP ... END LOOP;
```

Curseur et curseur paramétré.

```
DECLARE
    CURSOR cfilm is SELECT idfilm, titre FROM film;
    CURSOR cnotes(pidfilm NUMBER) IS SELECT nomint, note
    FROM notation NATURAL JOIN internaute WHERE idfilm = pidfilm;
BEGIN
    FOR cfilm_rec IN cfilm LOOP
      dbms_output.put_line('Notes de '||cfilm_rec.titre);
         FOR cnotes_r IN cnotes(cfilm_rec.idfilm) LOOP
           dbms_output.put_line(cnotes_r.nomint||' a note '||cnotes_r.note);
         END LOOP;
    END LOOP;
END;
```

Exception.

```
DECLARE
    vtitre film.titre%TYPE := :titre;
    vidfilm film.idfilm%TYPE;
    pasdenotes EXCEPTION;
    ... --def curseur
BEGIN
    SELECT idfilm INTO vidfilm FROM film WHERE titre = vtitre;
    SELECT count(*) INTO nbnotes FROM notation WHERE idfilm = vidfilm;
    IF nbnotes = 0 THEN
        RAISE pasdenotes;
    END IF;
    ... --parcours curseur
EXCEPTION
    WHEN no_data_found THEN  -- si le select ...into ne retourne aucun resultat
        dbms_output.put_line('Ce film n''est pas dans la base');
    WHEN too_many_rows THEN -- si le select ... into retourne plusieurs resultats
        dbms_output.put_line('Plusieurs films avec ce titre dans la base');
    WHEN pasdenotes THEN
        dbms_output.put_line('Films sans note');
    WHEN others THEN --cas par defaut : affiche les code et message de l'erreur
        dbms_output.put_line('Erreur : '||sqlcode||' '||sqlerrm);
END;
--autres exceptions : zero_divide, dup_val_on_index (INSERT avec une cle deja existante)
```

Procédure et fonction stockées.

```
CREATE OR REPLACE PROCEDURE maxentreepays (ppays VARCHAR2) IS
        vmax film.entree%type;
BEGIN
        SELECT max(entree) INTO vmax FROM film WHERE paysfilm=ppays;
        dbms_output.put_line(vmax);
END;

CREATE OR REPLACE FUNCTION fmaxentreepays (ppays VARCHAR2) RETURN NUMBER IS
        vmax film.entree%TYPE;
BEGIN
        SELECT max(entree) INTO vmax FROM film WHERE paysfilm=ppays;
        RETURN(vmax);
END;

CREATE OR REPLACE PROCEDURE iomax(ppays VARCHAR2, pmin OUT NUMBER, pmax OUT NUMBER) is
BEGIN
    SELECT min(entree), max(entree) INTO pmin, pmax FROM film WHERE paysfilm = ppays;
END;
```

Déclencheur.

```
CREATE [OR REPLACE] TRIGGER <Nom_Trigger>
[BEFORE | AFTER] [INSERT [OR] DELETE [OR] UPDATE [OF <Attribut>]] ON <Nom_Table>
[FOR EACH ROW][WHEN <Condition>]
DECLARE
BEGIN
[EXCEPTION]
END

CREATE OR REPLACE TRIGGER statnotation
    AFTER INSERT ON film FOR EACH ROW
DECLARE
    VNbInsert Number(3);
BEGIN
    SELECT NbInsert INTO VNbInsert FROM statbase WHERE NomTable='film';
    UPDATE statbase SET NbInsert = VNbInsert+1 WHERE NomTable='film' ;
EXCEPTION
    WHEN No_Data_Found THEN
       INSERT INTO statbase VALUES ('film', 1);
END;

CREATE OR REPLACE TRIGGER MajNbNotes
    AFTER INSERT ON notation
    FOR EACH ROW WHEN (new.note > 5) --Attention !!! pas de ":" ici
BEGIN
     UPDATE film SET nbnotes = nbnotes+1
     WHERE idfilm = :new.idfilm;  -- ":" avant new/old dans le begin
END;

CREATE OR REPLACE TRIGGER MajNbNotes
    AFTER INSERT OR DELETE ON notation FOR EACH ROW
BEGIN
    IF Inserting THEN
        UPDATE film SET nbnotes = nbnotes+1 WHERE idfilm = :new.idfilm;
    END IF ;
    IF Deleting THEN
        UPDATE film SET nbnotes = nbnotes-1 WHERE idfilm = :old.idfilm;
    END IF ;
END ;

CREATE OR REPLACE TRIGGER insertIdfilm
    BEFORE INSERT ON film
    FOR EACH ROW WHEN (new.idfilm = 0)
BEGIN
    --modif de la nouvelle valeur par le declencheur
    SELECT max(idfilm)+1 INTO :new.idfilm FROM film;
END;
```